DIAGRAM 01 / 0:00 - 3:10

Overall system

Ten-minute speaking script | 1.25 line spacing

[Point to Diagram 1: follow the browser-to-answer path, then the side connections.]

Today I will explain how Medix turns a medical question into a conversational answer with traceable support. The problem is that useful instructions may be scattered across medical videos, and copying transcript sentences does not necessarily answer someone's situation. Medix combines question understanding, evidence retrieval and language generation, with an optional video demonstration.

Starting at the top of the first diagram, the browser is the interface where users enter text or attach a photo and read the response. Next.js runs both the website and server-side application logic. Its /api/chat route receives the conversation, checks the session and coordinates the next step. The two-way connection to PostgreSQL represents saving and loading chat history; this database stores conversations rather than supplying medical facts.

The routing box decides whether to clarify the request, respond conversationally or retrieve evidence. It also uses history to resolve phrases such as "their circulation." A new medical question goes to FastAPI. A follow-up on the same situation may reuse the current evidence and go directly to the response composer, avoiding another full search.

FastAPI is the Python web service, not the language model. It exposes endpoints that invoke the retrieval and evidence-processing services. The catalog, transcript cache and local videos supply searchable metadata, timestamped speech and video material. FastAPI returns an evidence bundle: structured data containing selected transcript passages, cue identifiers, source and clip times, compact supported claims and an extracted clip location. This evidence response is an intermediate result, not necessarily the final wording the user reads.

The Next.js response composer is what I mean by the medical response layer. It is part of the same server, not another application. It combines relevant evidence with conversation context and checked NHS or St John Ambulance references. These references are curated, scope-limited summaries with source links, rather than a live search of the entire medical web. They support additional care guidance separately from the video.

The arrows to Voyager represent remote inference requests. Voyager is ASU's model gateway; Qwen is the model served through it. Python uses it for backend claims and, when needed, visual observations. Next.js uses it for conversational composition and verification. These calls use an existing model; they do not fine-tune it on our dataset.

Finally, the response is streamed to the browser after medical checks. A separate connection carries an optional relevant clip and transcript. The distinction matters: the answer explains what to do, while the evidence lets the user inspect why a claim is supported. A video should not appear simply because it shares broad medical keywords.

DIAGRAM 02 / 3:10 - 6:10

Text-only question

[Point to Diagram 2: trace the new-question branch, then the evidence-reuse branch.]

For the second diagram, consider: "I just put a bandage on someone's arm. How do I know if it's too tight?" First, Next.js validates the message and session. Intent routing separates a medical request from a greeting, an unclear question or an unrelated request. Conversation context helps preserve the situation without treating previous chat messages as medical evidence.

A new question is sent to POST /api/query on FastAPI. POST means sending a request body for the server to process; /api/query is the endpoint's address. The body contains the question as JSON. This is different from /api/chat, which is the Next.js entry point. One coordinates the conversation; the other runs the Python evidence pipeline.

Safety triage occurs before ordinary retrieval and can divert urgent or restricted requests. For normal retrieval, MiniLM converts text into numerical embeddings so meaning can be compared. Lexical matching also checks overlapping words. Together they rank catalog entries and transcript passages, instead of relying only on a video's title.

Next, the pipeline searches transcript windows to locate useful portions of candidate videos. CLIP compares the question with sampled video frames and contributes a visual relevance score. Even this text-only path can inspect retrieved video frames. CLIP is a matching model, not the assistant writing the answer, and its score is not a probability of medical correctness.

The extraction stage produces transcript cues, evidence frames and a local video clip. FFmpeg performs the media extraction. Cue timestamps preserve the connection between speech and the source video. Live retrieval estimates relevant boundaries; it does not automatically reproduce a dataset's annotated answer interval.

Voyager then generates structured backend claims from transcript cues. The application checks cue identifiers and supporting text. Extracted frames are evidence artifacts, but this backend text-generation step does not send the whole video to Qwen.

Back in Next.js, relevant cues and checked medical references support a conversational answer. Generation is followed by verification and a bounded repair attempt when needed. The aim is to answer directly, add practical detail proportionate to the question and keep each citation attached to the claim it supports.

For a follow-up such as "What if the colour doesn't come back quickly?", the reuse arrow keeps the current situation and evidence available. The response should address the next action rather than merely repeat the earlier threshold. The browser displays clean source links and clickable video timestamps where supported, alongside a relevant demonstration when available.

DIAGRAM 03 / 6:10 - 7:25

Image without a question

[Point to Diagram 3: emphasize the clarification branch and the two paths that are not executed.]

The third diagram describes a deliberate limitation of the current implementation. A photo alone does not trigger automatic diagnosis or procedure selection. The upload route accepts one supported JPEG or PNG within its size limit. The image is passed as inline encoded bytes, and the chat route checks whether the latest message contains a question.

When there is an image but no text, the system follows a deterministic clarification path. It asks: "What would you like help with in this photo?" The dashed connections mean no vision call and no retrieval run occur for that turn. This prevents the application from guessing what the user wants or treating appearance as proof of a medical condition.

The user can then provide a specific question, such as asking how to check whether a bandage is too tight. To enter the combined workflow, the image needs to be attached with that question. Currently, a later text-only reply does not automatically resend a photo from an earlier message: conversation history is retained, but image bytes are read from the latest message. The final arrow therefore leads to the text-plus-image diagram once both inputs are present.

DIAGRAM 04 / 7:25 - 10:00

Text plus image

[Point to Diagram 4: follow both inputs, then distinguish visual context from medical evidence.]

In the fourth diagram, the text and image meet at the Next.js routing step. The question establishes what help is needed, while history supplies context. A new image requests fresh retrieval rather than silently relying on an older evidence bundle. Next.js sends the question and encoded image bytes to POST /api/multimodal/query.

Pillow, the Python image library, validates the image, corrects orientation, converts its color format and resizes it. The normalized image sent to the model has its EXIF metadata removed. This step controls the input and reduces transmission size; it does not make the picture medically reliable.

Voyager Qwen receives the normalized photo and question for a constrained visual observation. It returns structured fields for visible objects, visible actions and uncertainty. The backend validates that structure. For example, an observation may identify a bandage, but a photograph cannot establish whether circulation is adequate.

Object and action descriptions become explicitly unverified retrieval hints. MiniLM and lexical search use them to help find relevant material. The separate arrow from the original question preserves the user's intent during localization and question answering, so incidental image details do not replace the actual question.

From there, CLIP scores retrieved video frames, temporal localization chooses candidate intervals, and extraction creates cues, frames and a clip. This is different from Qwen examining the uploaded photograph. Backend text generation produces supported claims, and Next.js combines relevant evidence, references and conversation context into the final answer.

Notice the dashed observation-to-generation arrow: image context can personalize the explanation, but it is not clinical evidence. Reference links support additional guidance; timestamp links support specific video statements. The frontend preserves playback, evidence moments and transcript synchronization without attaching unrelated moments just to fill the interface.

To conclude, Medix coordinates existing models and traceable evidence across these four paths. It is not a fine-tuned clinical model or a validated diagnostic system. Its value is connecting a user's question to an understandable answer and inspectable support. Source coverage, retrieval quality and model availability remain practical limits.